

Universidad de Costa Rica
Desarrollo de Software III
Proyecto Final Web

Práctica Guiada 2

Arquitectura MVC, conexión a MySQL, autenticación y primer CRUD funcional

1. Datos generales

Elemento	Descripción
Curso	Desarrollo de Software III
Tipo de actividad	Práctica guiada de avance del proyecto final
Modalidad	Trabajo grupal
Duración sugerida	3 horas
Tecnologías base	Java, JSP, Servlets, JDBC, MySQL, HTML5, CSS3 y Bootstrap o estilos propios
Producto esperado	Proyecto Java Web con estructura MVC inicial, conexión JDBC, login funcional, control de sesión, roles y primer CRUD conectado a MySQL

2. Propósito de la práctica

Esta práctica tiene como propósito que cada equipo convierta el diseño técnico elaborado en la primera práctica en una base funcional de aplicación web. La prioridad es que el proyecto deje de ser únicamente una propuesta visual y cuente con arquitectura organizada, conexión real a MySQL, inicio de sesión, protección de páginas y un primer módulo CRUD completo.

Al finalizar, cada equipo deberá demostrar que el sistema puede ejecutarse en un servidor de aplicaciones, conectarse a la base de datos, autenticar usuarios y administrar al menos una entidad principal del proyecto.

3. Objetivo general

Implementar la base funcional del proyecto web mediante una arquitectura MVC o limpia, integrando conexión a MySQL, autenticación, manejo de sesiones, control de roles y el primer CRUD completo del sistema.

4. Objetivos específicos

1. Crear la estructura inicial del proyecto bajo una arquitectura organizada por capas.

2. Configurar la conexión a MySQL mediante JDBC y consultas parametrizadas.
3. Implementar autenticación de usuarios con validación contra la base de datos.
4. Aplicar manejo de sesiones utilizando HttpSession.
5. Proteger páginas privadas y controlar el acceso según el rol del usuario.
6. Implementar el primer módulo CRUD completo conectado a MySQL.
7. Registrar evidencias técnicas de funcionamiento para la documentación final.

5. Contexto de trabajo

En la práctica anterior, cada equipo definió el problema, usuarios, roles, módulos, requerimientos y base de datos preliminar. En esta segunda práctica, ese diseño se lleva a código. El trabajo debe respetar la separación entre presentación, controladores, lógica de negocio y acceso a datos.

No se permite resolver el sistema colocando toda la lógica dentro de páginas JSP. Las páginas JSP deben utilizarse principalmente para mostrar información y formularios; los Servlets deben recibir y procesar solicitudes; los DAO deben concentrar el acceso a la base de datos; y los modelos deben representar las entidades del sistema.

6. Actividad 1: Preparación del entorno de desarrollo

Cada equipo debe verificar que el entorno esté listo antes de programar. La práctica no debe avanzar si el proyecto no puede ejecutarse localmente o si no existe una base de datos disponible para pruebas.

Checklist mínimo

- IDE configurado: NetBeans, IntelliJ IDEA, Eclipse o Visual Studio Code con soporte para Java Web.
- Servidor de aplicaciones disponible: Apache Tomcat, GlassFish u otro compatible.
- MySQL instalado o disponible mediante un servidor local/remoto.
- Driver JDBC de MySQL agregado al proyecto.
- Repositorio Git actualizado y compartido con todos los integrantes.
- Carpeta database con schema.sql y data.sql actualizados.
- README inicial con instrucciones para ejecutar el proyecto.

Producto de la actividad

Elemento revisado	Estado	Evidencia o comentario
Servidor de aplicaciones	Completado	El proyecto utiliza Spring Boot con servidor Apache Tomcat embebido. La aplicación se ejecuta localmente desde la clase principal CarpinteriaCircularApplication.java y se accede mediante http://localhost:8080 .
Base de datos MySQL	Completado	El sistema se encuentra conectado a una base de datos MySQL real. La conexión permite almacenar y consultar información de productos, clientes, pedidos, detalles de pedidos, envíos, usuarios y demás entidades del sistema.
Driver JDBC	Completado	El proyecto incluye el conector de MySQL configurado mediante Maven. La

Elemento revisado	Estado	Evidencia o comentario
		comunicación con la base de datos se realiza a través de Spring Data JPA, Hibernate y el driver JDBC de MySQL.
Repositorio Git	Completado	El proyecto se encuentra publicado en el repositorio público granadosJefferson/CarpinteriaCircularWeb. El repositorio contiene el código fuente, configuración Maven, documentación, scripts de base de datos y recursos del sistema.
Scripts SQL	Completado	El repositorio contiene la carpeta database con los archivos schema.sql y data.sql, utilizados como respaldo de la estructura de la base de datos y de los datos iniciales del sistema.
README inicial	Pendiente	Está pendiente de crearse en esta Segunda etapa de la guía del Proyecto.

7. Actividad 2: Creación de la estructura MVC del proyecto

Cada equipo debe organizar el código del proyecto siguiendo una estructura clara. La organización puede variar según el IDE, pero debe evidenciar separación de responsabilidades.

Estructura sugerida

```

nombre-proyecto/
|-- src/main/java/
|   |-- model/           Entidades del sistema
|   |-- dao/             Acceso a datos mediante JDBC
|   |-- service/         Reglas de negocio o validaciones de aplicación
|   |-- servlet/         Controladores web
|   |-- util/            Conexión, constantes y utilidades
|   |-- filter/          Filtros de seguridad, si se utilizan
|-- src/main/webapp/
|   |-- views/           Páginas JSP privadas
|   |-- auth/            JSP de login o recuperación
|   |-- css/             Estilos
|   |-- js/              Scripts
|   |-- index.jsp
|-- database/
|   |-- schema.sql
|   |-- data.sql
|-- README.md

```

Producto de la actividad

Capa o paquete	Responsabilidad	Clase o archivo creado
model	Representa las entidades principales del sistema y define su relación con las tablas de MySQL mediante anotaciones JPA.	Usuario.java, Producto.java, Cliente.java, Pedido.java, DetallePedido.java, Envio.java, EstadoPedido.java, EstadoEnvio.java, MetodoPago.java, TipoEntrega.java, CheckoutFormulario.java, PedidoFormulario.java e ItemPedidoFormulario.java.
dao	Contiene las operaciones de acceso a MySQL. En Spring Boot esta capa se implementa mediante el paquete repository, utilizando Spring Data JPA en lugar de consultas JDBC manuales.	UsuarioRepository.java, ProductoRepository.java, ClienteRepository.java, PedidoRepository.java, DetallePedidoRepository.java y EnvioRepository.java.
service	Contiene las reglas de negocio, validaciones y coordinación entre los repositorios. Se encarga de procesos como autenticación, pedidos, control de existencias, pagos, envíos y administración de productos y clientes.	UsuarioService.java, ProductoService.java, ClienteService.java, PedidoService.java, EnvioService.java y CheckoutService.java.
servlet	Recibe las solicitudes HTTP, procesa las acciones del usuario y redirige o envía información a las vistas. En Spring Boot esta función se implementa mediante clases del paquete controller.	InicioController.java, AuthController.java, ProductoController.java, ClienteController.java, PedidoController.java, EnvioController.java, CheckoutController.java, DashboardController.java y ReporteController.java.
views / JSP	Muestra formularios, tablas, detalles, mensajes y datos al usuario. En el proyecto se utiliza Thymeleaf en lugar de páginas JSP.	index.html, login.html, registro.html, perfil.html, catalogo.html, carrito.html, checkout.html, compra-exitosa.html, dashboard.html, admin/productos.html, admin/producto-formulario.html, admin/clientes.html, admin/cliente-formulario.html, admin/pedidos.html, admin/pedido-formulario.html, admin/pedido-detalle.html, admin/envios.html, admin/envio-formulario.html,

Capa o paquete	Responsabilidad	Clase o archivo creado
		admin/envio-detalle.html, admin/envio-editar.html y admin/reportes.html.
util	Contiene configuraciones y utilidades generales. En Spring Boot la conexión a MySQL no se administra mediante una clase JDBC manual, sino mediante la configuración del proyecto y las dependencias de Maven.	application.properties, pom.xml, DatosInicialesConfig.java y CarpinteriaCircularApplication.java.

Debido a que el proyecto fue desarrollado con Spring Boot, algunos paquetes utilizan nombres diferentes a los indicados en la arquitectura tradicional. El paquete repository cumple la función de la capa DAO, los Controller sustituyen a los Servlets y las plantillas HTML con Thymeleaf sustituyen a las páginas JSP. La conexión con MySQL se configura mediante application.properties, Spring Data JPA, Hibernate y el driver JDBC incluido en Maven.

8. Actividad 3: Configuración de conexión a MySQL con JDBC

Cada equipo debe crear una clase responsable de abrir la conexión a la base de datos. Esta clase debe estar en un paquete de utilidad y debe reutilizarse desde los DAO. No se deben abrir conexiones directamente desde las páginas JSP.

El proyecto Carpintería Circular 360 utiliza Spring Boot, Spring Data JPA, Hibernate y el driver JDBC de MySQL para conectarse a la base de datos.

El equivalente a esta conexión del paquete de utilidades es la conexión de las propiedades de la aplicación o src/main/resources/application.properties el cual almacena la conexión con MySQL y se conecta a repository que sería el dao.

A diferencia de una aplicación Java Web tradicional, no fue necesario crear una clase manual llamada DBConnection.

En este archivo se definen los datos necesarios para que Spring Boot establezca la conexión con MySQL:

```
spring.datasource.url=jdbc:mysql://localhost:8080/nombre_base_datos
```

```
spring.datasource.username=usuario
```

```
spring.datasource.password=contraseña
```

```
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
```

El driver JDBC de MySQL se encuentra agregado como dependencia en el archivo:

```
pom.xml
```

El acceso a los datos se realiza mediante interfaces del paquete repository. Estas interfaces utilizan Spring Data JPA para ejecutar operaciones de consulta, inserción, actualización y eliminación sobre MySQL.

Ejemplo base de clase de conexión

```
package util;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class DBConnection {
    private static final String URL = "jdbc:mysql://localhost:3306/nombre_bd";
    private static final String USER = "root";
    private static final String PASSWORD = "";

    public static Connection getConnection() throws SQLException {
        return DriverManager.getConnection(URL, USER, PASSWORD);
    }
}
```

El equipo debe adaptar el nombre de la base de datos, usuario y contraseña según su entorno. Se recomienda evitar colocar credenciales reales en repositorios públicos.

Producto de la actividad

- Clase DBConnection o equivalente creada.
- Prueba de conexión ejecutada correctamente.
- Captura de pantalla o evidencia del servidor ejecutando el proyecto.
- Confirmación de que el proyecto se conecta a la base de datos definida en schema.sql.

Evidencias producto de la actividad:

1. Archivo application.properties

```
spring.application.name=carpinteria-circular

spring.datasource.url=jdbc:mysql://localhost:3306/carpinteria_circular
spring.datasource.username=root
spring.datasource.password=Contraseña

spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

spring.servlet.multipart.max-file-size=10MB
spring.servlet.multipart.max-request-size=10MB
```

app.upload.dir=uploads/productos

2. Spring boot iniciado correctamente.

```
2026-06-24T21:08:57.843-06:00 WARN 12000 --- [carpinteria-circular] [ restartedMain] JpaBaseConfiguration$JpaWebConfiguration : spring.jpa.open-in-view is enabled by default. The
refo, database queries may be performed during view rendering. Explicitly configure spring.jpa.open-in-view to disable this warning
2026-06-24T21:08:57.892-06:00 INFO 12000 --- [carpinteria-circular] [ restartedMain] o.s.b.w.a.WelcomePageHandlerMapping : Adding welcome page template: index
2026-06-24T21:08:58.869-06:00 INFO 12000 --- [carpinteria-circular] [ restartedMain] o.s.boot.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context pa
th '/'
2026-06-24T21:08:58.885-06:00 INFO 12000 --- [carpinteria-circular] [ restartedMain] c.c.CarpinteriaCircularApplication : Started CarpinteriaCircularApplication in 14.466 s
econds (process running for 15.728)
```

3. Pantalla del sistema iniciado mostrando datos desde MySQL.

Cientes encontrados: 4 ⓘ Los clientes inactivos se conservan para mantener el historial.

CLIENTE	CORREO	TELÉFONO	DIRECCIÓN	ESTADO	ACCIONES
D David Cliente CL-4	david@gmail.com	Pendiente	No indicada	Activo	Editar Desactivar
D David Venegas Cliente CL-3	davidvenegas@gmail.com	8984-7626	San Ramon, Alajuela, CR	Activo	Editar Desactivar
J Jefferson Granados Cliente CL-1	granadosyefi@gmail.com	72833238	Las brisas	Activo	Editar Desactivar
M María Pérez Cliente CL-2	maria@gmail.com	88446572	San ramon	Activo	Editar Desactivar

4. MySQL Workbench, tablas y registros del proyecto.

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

Tables_in_carpinteria_circular

- clientes
- detalle_pedido
- envios
- pedidos
- productos
- usuarios

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: |

	id	activo	correo	direccion	fecha_registro	nombre
▶	1	1	granadosyefi@gmail.com	Las brisas	2026-06-21 01:52:15.865296	Jefferson Granados
	2	1	maria@gmail.com	San ramon	2026-06-21 19:07:50.692185	María Pérez
	3	1	davidvenegas@gmail.com	San Ramon, Alajuela, CR	2026-06-21 19:24:59.905559	David Venegas
	4	1	david@gmail.com	NULL	2026-06-23 21:24:40.745967	David
*	NULL	NULL	NULL	NULL	NULL	NULL

Result Grid | Form Editor

9. Actividad 4: Implementación de autenticación y sesiones

Cada equipo debe implementar un inicio de sesión funcional. Las credenciales deben validarse contra la tabla de usuarios en MySQL. El sistema debe crear una sesión cuando el usuario inicia correctamente y destruirla cuando cierra sesión.

Elementos mínimos del login

- Página login.jsp o equivalente con formulario de correo y contraseña.

- Servlet LoginServlet que reciba la solicitud POST.
- UsuarioDAO con método para buscar usuario por credenciales.
- Uso de HttpSession para guardar usuario autenticado y rol.
- Servlet LogoutServlet para cerrar sesión.
- Redirección a una página privada cuando el login sea correcto.
- Mensaje de error cuando las credenciales sean inválidas.

Flujo esperado

```

Usuario ingresa correo y contraseña
    |
    v
LoginServlet recibe la solicitud POST
    |
    v
UsuarioDAO consulta MySQL con PreparedStatement
    |
    v
Si existe el usuario: crear HttpSession y redirigir al panel
Si no existe: volver al login con mensaje de error
  
```

Producto de la actividad

Elemento	Estado	Evidencia
Formulario de login	Completado	rc/main/resources/templates/login.html. Contiene los campos de credenciales y envía la solicitud al controlador de autenticación.
LoginServlet	Completado mediante equivalente	AuthController.java recibe y procesa la solicitud POST del inicio de sesión.
UsuarioDAO	Completado mediante equivalente	UsuarioRepository.java realiza el acceso a los usuarios almacenados en MySQL.
HttpSession	Completado	El usuario autenticado se guarda en la sesión con el atributo usuarioLogueado, junto con la información requerida para controlar su acceso.
LogoutServlet	Completado mediante equivalente	La ruta /logout, implementada en AuthController.java, invalida la sesión activa.
Mensajes de error	Completado	Los administradores son enviados al menú administrativo y los clientes a su perfil o a la ruta solicitada.
Servicio de autenticación	Completado	UsuarioService.java valida las credenciales y obtiene el usuario correspondiente.

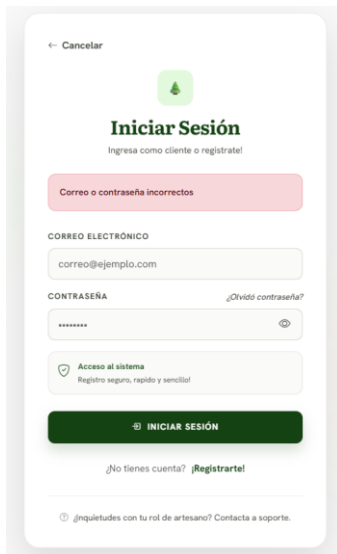
Elemento	Estado	Evidencia
Redirección a página privada	Completado	Los administradores son enviados al menú administrativo y los clientes a su perfil o a la ruta solicitada.
Control de roles	Completado	El sistema distingue entre usuarios con rol ADMIN y usuarios con rol CLIENTE.
Protección de funciones privadas	Completado	Las funciones administrativas y el carrito requieren que exista un usuario autenticado en la sesión.

Evidencias de uso

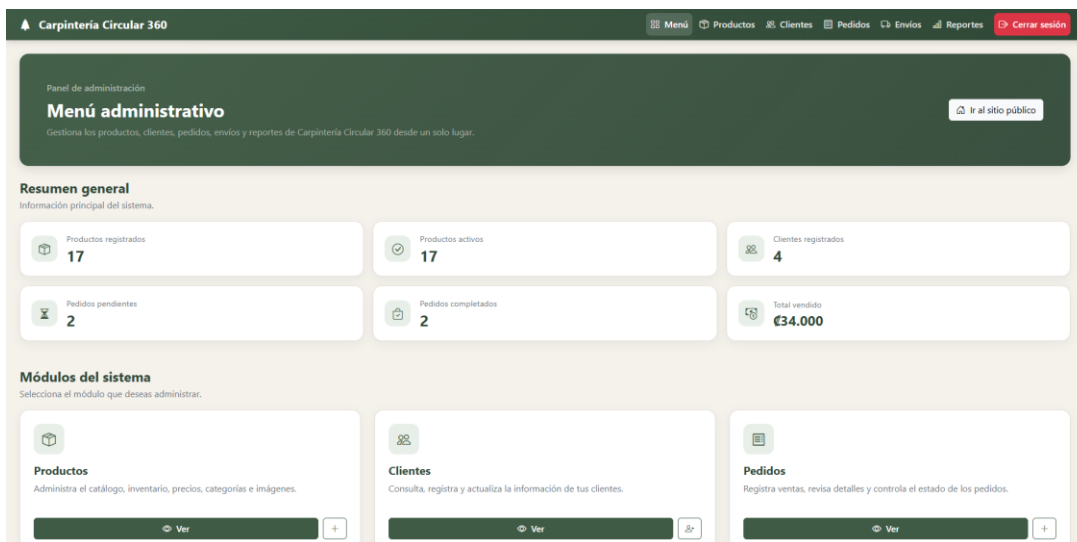
1. Captura de pantalla del formulario de inicio de sesión.



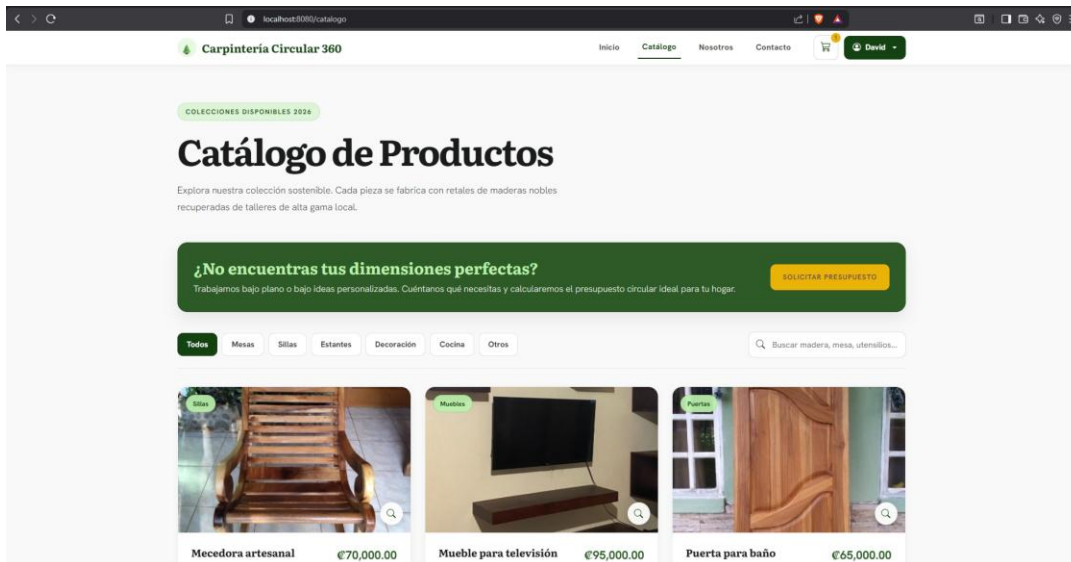
2. Captura de un intento con credenciales inválidas.



3. Captura del menú administrativo después de iniciar sesión como administrador.

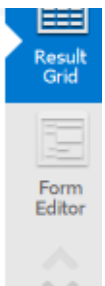


4. Captura del perfil después de iniciar sesión como cliente.



5. Captura de la tabla de usuarios en MySQL, ocultando las contraseñas.

	id	correo	nombre	password	rol
▶	3	admin@carpinteria.com	Administrador	\$2a\$10\$9Dqq4tAetccfoYPogmu7yeNqCwafLTjs...	ADMIN
	4	cliente@correo.com	Cliente Prueba	\$2a\$10\$AecITvO.38d6rVWSQzypAu4gfm3hq4...	CLIENTE
	5	michizaespi@gmail.com	michell zamora	\$2a\$10\$Fn1Ys58BFoiwut2NCyaK8.En47w0RJlk/...	CLIENTE
	6	registro@gmail.com	registroNuevo	\$2a\$10\$m72emS2hzULQk59g6CblNOEw9nVVTV...	CLIENTE
	7	david@gmail.com	David	\$2a\$10\$83PntauXQ1HElquCdLfbwutLLjcpBqrfi...	CLIENTE
★	NULL	NULL	NULL	NULL	NULL



10. Actividad 5: Protección de páginas privadas y control de roles

El sistema debe impedir que un usuario no autenticado acceda directamente a páginas privadas copiando la URL en el navegador. Además, los roles definidos por el equipo deben tener accesos diferenciados según sus responsabilidades.

El sistema Carpintería Circular 360 aplica protección de páginas privadas mediante el uso de HttpSession y validaciones de rol en los controladores de Spring Boot.

Cuando un usuario inicia sesión correctamente, el sistema guarda el usuario autenticado en la sesión mediante el atributo:

```
session.setAttribute("usuarioLogueado", usuario);
```

Los controladores de los módulos administrativos verifican que dicho atributo exista y que el usuario tenga el rol ADMIN. Si el usuario no ha iniciado sesión o no posee permisos suficientes, se le redirige al formulario de inicio de sesión o a una página permitida.

La validación se realiza en el backend, por lo que no es suficiente copiar directamente una dirección administrativa en el navegador. Aunque una opción no aparezca en el menú, el controlador vuelve a comprobar la sesión y el rol antes de mostrar o modificar información.

Acciones requeridas

- Crear una validación de sesión en cada Servlet privado o implementar un filtro de autenticación.
- Redirigir al login cuando no exista una sesión activa.
- Ocultar o deshabilitar opciones de menú que no correspondan al rol del usuario.
- Validar permisos también en el backend, no solo en la interfaz.
- Probar al menos dos usuarios con roles diferentes.

Matriz de acceso

Módulo o página	Administrador	Usuario regular	Observaciones
Panel principal	Permitido	No permitido	El administrador puede ingresar al panel mediante /admin. El cliente es dirigido a su perfil o a las páginas públicas.
Gestión de usuarios	Parcialmente permitido	Acceso a registro, login y perfil propio	El sistema permite registrar usuarios, iniciar sesión, manejar roles y consultar el perfil. No existe todavía un CRUD administrativo completo de cuentas de usuario.
Primer CRUD del proyecto	Acceso completo	Consulta limitada	El administrador administra productos. El cliente puede consultar los productos activos en el catálogo.
Reportes o consultas	Permitido	No permitido	El administrador puede acceder al módulo de reportes y consultar métricas del sistema.
Configuración o mantenimiento	Parcialmente permitido	No permitido	No existe un módulo llamado "Configuración". Sin embargo, el administrador realiza mantenimiento mediante los CRUD de productos, clientes, pedidos y envíos.

11. Actividad 6: Implementación del primer CRUD funcional

Cada equipo debe seleccionar una entidad principal del proyecto e implementar un CRUD completo. Se recomienda iniciar con una entidad central, pero no excesivamente compleja. Por ejemplo: productos, clientes, libros, citas, solicitudes, cursos, eventos, empleados, recursos o incidencias.

Para el primer CRUD funcional se seleccionó la entidad Producto, debido a que representa uno de los elementos principales del sistema Carpintería Circular 360.

El módulo permite registrar productos, consultar los existentes, editar su información, controlar sus existencias y desactivarlos sin eliminar permanentemente sus datos.

Operaciones mínimas del CRUD

- Registrar un nuevo elemento.
- Listar los elementos existentes.
- Consultar el detalle de un elemento.
- Editar un elemento.
- Eliminar o desactivar un elemento.
- Mostrar mensajes de éxito, advertencia o error.
- Validar campos obligatorios desde el servidor.
- Utilizar PreparedStatement para las consultas SQL.

Estructura recomendada para el CRUD

Componente	Responsabilidad	Ejemplo
Modelo	Representa la entidad Producto y su relación con la tabla correspondiente en MySQL.	Producto.java
DAO	Realiza las operaciones de persistencia. En Spring Boot esta función es realizada por Spring Data JPA.	ProductoRepository.java
Servicio	Contiene las validaciones y reglas de negocio relacionadas con productos.	ProductoService.java
Servlet	Recibe las solicitudes HTTP del módulo. En Spring Boot esta función corresponde al controlador.	ProductoController.java
Vista de Listado	Muestra la tabla de productos registrados y las opciones administrativas.	templates/admin/productos.html
Vista de formulario	Permite registrar y editar productos.	templates/admin/producto-formulario.html
Vista de detalle	Muestra al usuario la información completa del producto mediante una vista rápida.	templates/catalogo.html, mediante modal de detalle
Recurso de imágenes	Almacena las imágenes asociadas con los productos.	uploads/productos/

Funcionamiento del CRUD

Registrar

El administrador puede ingresar los datos de un nuevo producto mediante el formulario administrativo. Al guardar, la información es validada y posteriormente almacenada en MySQL.

Listar

El sistema consulta los productos registrados y los muestra en una tabla administrativa. También muestra los productos activos en el catálogo público.

Consultar detalle

El catálogo cuenta con una vista rápida que muestra:

- imagen;
- nombre;
- categoría;
- precio;
- descripción;
- existencias;
- opción para agregar el producto al carrito;
- opción para consultar mediante WhatsApp.

Editar

El administrador puede seleccionar un producto existente, cargar sus datos en el formulario y modificar su información.

Eliminar o desactivar

Los productos se desactivan en lugar de eliminarse permanentemente. Esto permite conservar información histórica relacionada con pedidos y ventas.

Mensajes

El sistema muestra mensajes de éxito o error después de las operaciones realizadas por el administrador.

Acceso a la base de datos

El proyecto no utiliza consultas JDBC escritas manualmente con PreparedStatement. La persistencia se realiza mediante ProductoRepository, Spring Data JPA e Hibernate.

Estas tecnologías generan consultas SQL parametrizadas y gestionan internamente las operaciones equivalentes a:

- INSERT;
- SELECT;
- UPDATE;

- desactivación lógica.

Producto de la actividad

Operación	Ruta o archivo	Funciona correctamente	Observaciones
Registrar	/admin/productos/nuevo y producto-formulario.html	Sí	Permite ingresar los datos y guardar un nuevo producto en MySQL.
Listar	/admin/productos y productos.html	Sí	Muestra los productos existentes en una tabla administrativa.
Consultar detalle	/catalogo y modal dentro de catalogo.html	Sí	La información completa del producto se muestra mediante una vista rápida en el catálogo.
Editar	Ruta administrativa de edición y producto-formulario.html	Sí	El formulario carga la información existente y permite actualizarla.
Eliminar o desactivar	Acción administrativa desde productos.html	Sí	Se utiliza desactivación lógica para conservar la información histórica.
Validaciones	ProductoService.java, Producto.java y formulario HTML	Sí	Se validan los datos antes de guardar o actualizar el registro.

12. Actividad 7: Validaciones, mensajes y manejo básico de errores

El primer CRUD debe incluir validaciones mínimas. No se aceptará como funcional un formulario que permita guardar datos vacíos, inconsistentes o con errores evidentes. Las validaciones deben realizarse en el servidor, aunque también pueden reforzarse con JavaScript o atributos HTML5.

El primer CRUD seleccionado corresponde al módulo de productos. Este módulo incorpora validaciones tanto en el servidor como en el formulario HTML para evitar el registro de datos vacíos, inválidos o inconsistentes.

En el backend, las validaciones se realizan principalmente desde ProductoService.java, ProductoController.java y la entidad Producto.java. En el frontend, se refuerzan mediante atributos HTML5 y mensajes visuales en las plantillas Thymeleaf.

Validaciones sugeridas

Tipo de validación	Ejemplo	Lugar mínimo requerido
Campo obligatorio	Nombre, descripción, fecha, correo o estado.	Servidor
Formato	Correo electrónico, teléfono o fecha válida.	Servidor y cliente
Longitud	Evitar textos demasiado cortos o demasiado largos.	Servidor
Integridad	No registrar una relación con un ID inexistente.	Servidor
Duplicados	Evitar correos, códigos o nombres únicos repetidos.	Servidor

Validaciones aplicadas

Tipo de validación	Ejemplo aplicado en el proyecto	Lugar mínimo requerido
Campo obligatorio	Nombre, descripción, categoría, precio y cantidad del producto.	Servidor y cliente
Formato	Precio y cantidad deben contener valores numéricos válidos.	Servidor y cliente
Longitud	Se limita la cantidad de caracteres permitidos en nombre, descripción, categoría e imagen.	Servidor y cliente
Integridad	Antes de editar, activar o desactivar un producto, se verifica que el identificador exista en MySQL.	Servidor
Duplicados	Se pueden validar productos repetidos según los criterios definidos por el sistema, por ejemplo el nombre.	Servidor
Precio	El precio debe ser mayor o igual que cero.	Servidor
Existencias	La cantidad disponible no puede ser negativa.	Servidor
Estado	El producto debe conservar un estado válido, como activo o inactivo.	Servidor

Producto de la actividad

- Mensajes de éxito al guardar, editar o eliminar.
- Mensajes de error cuando falten datos obligatorios.
- Manejo básico de excepciones en DAO o Servlet.
- Validación de datos antes de ejecutar INSERT o UPDATE.
- Evitar mostrar errores técnicos directamente al usuario final.

13. Actividad 8: Prueba guiada del avance

Cada equipo debe ejecutar una prueba corta del avance para verificar que la base funcional del sistema está lista para continuar con los siguientes módulos.

Caso de prueba	Resultado esperado	Resultado obtenido
Ingresar con usuario válido	El sistema crea sesión y muestra el panel principal.	Correcto. El usuario administrador accede al panel administrativo y el cliente accede a las opciones correspondientes a su rol.
Ingresar con usuario inválido	El sistema muestra mensaje de error.	Correcto. El sistema rechaza las credenciales inválidas y muestra un mensaje de error.
Acceder a página privada sin sesión	El sistema redirige al login.	Correcto. Las rutas privadas verifican la sesión y redirigen al formulario de inicio de sesión.
Acceder con rol restringido	El sistema limita opciones o bloquea el acceso.	Correcto. El usuario con rol CLIENTE no puede acceder a las rutas administrativas.
Registrar dato en el CRUD	El dato se guarda en MySQL.	Correcto. El producto se registra y aparece en el listado administrativo.
Editar dato en el CRUD	El dato se actualiza correctamente.	Correcto. Los cambios realizados al producto se almacenan en MySQL.
Eliminar o desactivar dato	El dato se elimina o cambia de estado.	Correcto. El producto se desactiva y deja de mostrarse como disponible en el catálogo.
Enviar formulario incompleto	El sistema muestra validación.	Correcto. El sistema impide guardar datos incompletos o inválidos y muestra el mensaje correspondiente.

14. Entregables de la práctica

Al finalizar la práctica, cada equipo debe subir al repositorio o entregar las siguientes evidencias:

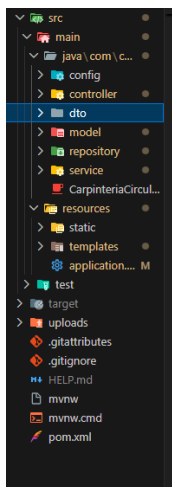
1. Proyecto Java Web ejecutándose correctamente en el servidor de aplicaciones.

```
... Run: CarpinteriaCircularApplication + ~ [ ] [ ] | [ ] X
r] [ restartedMain] c.c.CarpinteriaCircularApplication : S
tarted CarpinteriaCircularApplication in 13.984 seconds (process
running for 15.188)
2026-06-24T21:40:30.835-06:00 INFO 1932 --- [carpinteria-circula
r] [nio-8080-exec-1] o.a.c.c.C.[Tomcat].[localhost].[/] : I
nitializing Spring DispatcherServlet 'dispatcherServlet'
2026-06-24T21:40:30.838-06:00 INFO 1932 --- [carpinteria-circula
r] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : I
nitializing Servlet 'dispatcherServlet'
2026-06-24T21:40:30.842-06:00 INFO 1932 --- [carpinteria-circula
r] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : C
ompleted initialization in 2 ms
```



El proyecto se ejecuta correctamente mediante Spring Boot y el servidor Tomcat embebido en el Puerto 8080.

2. Estructura MVC o limpia creada en el proyecto.



El proyecto está organizado por capas, separando controladores, servicios, repositorios, modelos, configuración y vistas.

3. Clase de conexión JDBC funcionando.

```

3  spring.datasource.url=jdbc:mysql://localhost:3306/carpinteria_circular
4  spring.datasource.username=protegido para captura
5  spring.datasource.password=protegido para captura
6

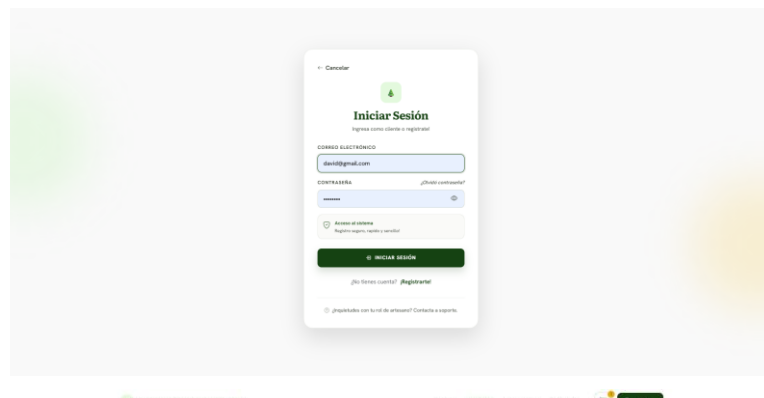
```

Pom.xml

```
<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <scope>runtime</scope>
</dependency>
```

La conexión con MySQL se configura mediante Spring Boot en application.properties y utiliza el driver JDBC incluido en Maven.

4. Login funcional conectado a MySQL.



The top part of the image shows a mobile app login screen titled 'Iniciar Sesión'. It includes fields for 'CORREO ELECTRONICO' (david@gmail.com) and 'CONTRASEÑA', a 'Recordarme' checkbox, and an 'Iniciar Sesión' button. Below the login screen is a web browser view of a catalog for 'Carpintería Circular 360'. The catalog features a 'Catálogo de Productos' section with a search bar and filters. It displays three products: 'Mecedora artesanal' (€70,000.00), 'Mueble para televisión' (€95,000.00), and 'Puerta para baño' (€65,000.00).

id	correo	nombre	password	rol
7	david@gmail.com	David	\$2a\$10\$83PntauXQ1HElquCdLfBwutLLjcpBqrfl...	CLIENTE
	NULL	NULL	NULL	NULL

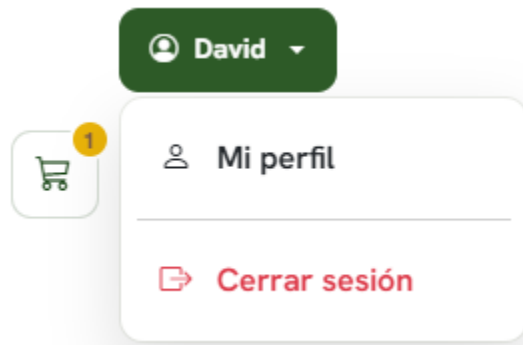
5. Manejo de sesión con HttpSession.

```

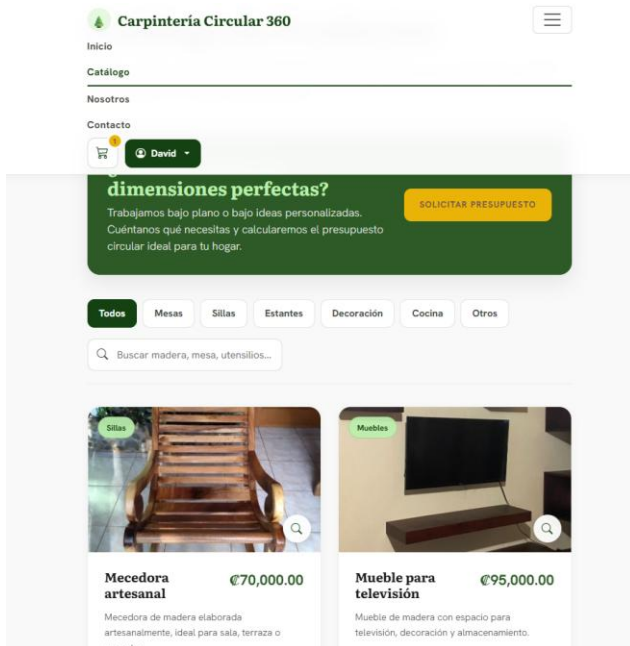
318 private void guardarUsuarioEnSession(
319     HttpSession session,
320     Usuario usuario) {
321
322     session.setAttribute(
323         "usuarioLogueado",
324         usuario
325     );
326
327     session.setAttribute(
328         "correo",
329         usuario.getCorreo()
330     );
331
332     session.setAttribute(
333         "nombre",
334         usuario.getNombre()
335     );
336
337     session.setAttribute(
338         "rol",
339         usuario.getRol()
340     );
341 }

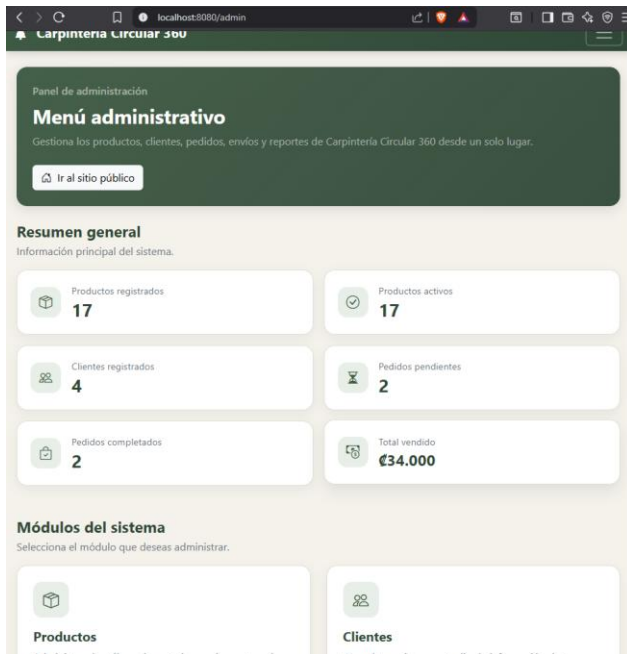
```

6. Cierre de sesión funcional.



7. Control básico de roles.





8. Primer CRUD completo conectado a MySQL.

Información del producto

Nombre del producto *

Utiliza un nombre claro y fácil de identificar.

Descripción

Mecedora de madera elaborada artesanalmente, ideal para sala, terraza o corredor.

Precio *

€ 70000.00

Cantidad disponible *

6

Categoría *

Sillas

Imagen del producto

Seleccionar imagen

Seleccionar archivo Sin archivos seleccionados

Deja el campo vacío para conservar la imagen actual.

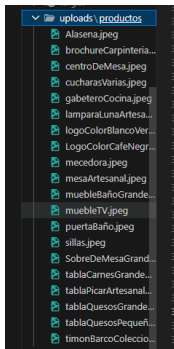
[Cancelar](#)
[Guardar cambios](#)

Result Grid								
Filter Rows:								
	id	activo	cantidad	categoria	descripcion	imagen	nombre	precio
▶	2	1	6	Sillas	Mecedora de madera elaborada artesanalmente...	meecedora.jpeg	Mecedora artesanal mediana	70000.00
✱	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

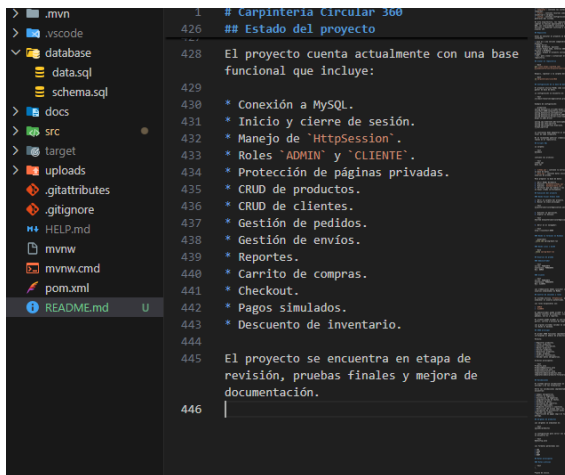
9. Validaciones básicas del primer CRUD.

Que no esté vacío, requiere al menos llenar 4 campos.

10. Capturas de pantalla del avance.



11. README actualizado con instrucciones de ejecución.



12. Scripts schema.sql y data.sql actualizados.

La base de datos se crea con schema.sql: Por verificar data.sql contiene usuarios de prueba: Sí

Actualmente ya tengo la base de datos de Carpinteria_circular creada, antes de trabajar en las guías, una nueva precarga de datos o creacion de tablas podría generar duplicación de datos o inconsistencias en la

Compatibilidad de espacios entre las tablas creadas en el schema.sql ya que con el Proyecto y la logica que se maneja usamos hibernate y esto cambia, elimina y agrega nuevos espacios en las tablas.

15. Criterios de evaluación

Criterio	Puntaje
Configura correctamente el entorno, servidor, base de datos y repositorio.	10 pts
Organiza el proyecto bajo una arquitectura MVC o limpia.	15 pts
Implementa conexión JDBC funcional y reutilizable.	15 pts
Implementa login, logout y manejo de sesión.	15 pts
Aplica control básico de roles y protección de páginas privadas.	10 pts
Desarrolla un primer CRUD completo conectado a MySQL.	25 pts
Incluye validaciones, mensajes y manejo básico de errores.	5 pts
Actualiza README, scripts SQL y evidencias del avance.	5 pts
Total	100 pts

16. Preguntas de cierre

Cada equipo debe responder brevemente las siguientes preguntas al finalizar la práctica:

1. ¿Cuál fue la mayor dificultad técnica durante la implementación?

La mayor dificultad fue la documentación del proyecto, especialmente adaptar la guía tradicional de JSP, Servlets y DAO a la arquitectura utilizada con Spring Boot, Thymeleaf, controladores, servicios y repositorios.

2. ¿Qué parte del proyecto ya está realmente conectada a la base de datos?

Actualmente están conectados a MySQL los módulos de usuarios, clientes, productos, pedidos, detalles de pedidos y envíos. El sistema permite registrar, consultar, editar y actualizar información real almacenada en la base de datos.

3. ¿El login valida credenciales reales desde MySQL?

Sí. El sistema consulta los usuarios almacenados en MySQL mediante UsuarioRepository y UsuarioService. Cuando las credenciales son correctas, se crea una sesión con HttpSession; si son incorrectas, se muestra un mensaje de error.

4. ¿Qué rol tiene más permisos y qué rol tiene menos permisos?

El rol ADMIN tiene más permisos, porque puede acceder al panel administrativo y gestionar productos, clientes, pedidos, envíos y reportes.

El rol CLIENTE tiene menos permisos y puede acceder principalmente al catálogo, perfil, carrito y proceso de compra.

5. ¿Cuál fue el primer CRUD implementado y por qué se eligió?

El primer CRUD implementado fue el de productos. Se eligió porque Producto es una entidad central del sistema y se relaciona directamente con el catálogo, el inventario, los pedidos y las ventas.

6. ¿Qué validaciones faltan por mejorar?

Se pueden mejorar las validaciones de productos duplicados, longitud de textos, formato de teléfonos, seguridad de contraseñas, tamaño y tipo de imágenes, manejo global de excepciones y mensajes específicos para cada campo del formulario.

7. ¿Qué módulo se implementará en la tercera práctica guiada?

En la tercera práctica se continuará con la revisión y fortalecimiento de los módulos restantes, principalmente pedidos, envíos, reportes y validaciones generales.

Además, se completarán las pruebas integrales, la documentación técnica, las evidencias y la preparación final del sistema para su entrega y defensa.

17. Resultado esperado

Al final de esta práctica, cada equipo debe contar con la base técnica del proyecto I del proyecto final. El proyecto debe ejecutarse, conectarse a MySQL, permitir inicio y cierre de proyecto, proteger páginas privadas, diferenciar al menos dos roles y administrar una entidad mediante un CRUD de proyecto.

Esta práctica servirá como base para la siguiente etapa, en la cual se completarán los CRUD restantes, se incorporarán proyectos o consultas, se reforzarán validaciones, se documentarán pruebas y se preparará la defensa final del proyecto.

18. Checklist rápido antes de salir del laboratorio

Verificación final	Sí / No	Comentario
El proyecto compila y despliega correctamente.	Sí	La aplicación se ejecuta correctamente con Spring Boot y el servidor Tomcat embebido.
La base de datos se crea con schema.sql.	Por verificar	El archivo existe, pero no se ha probado recientemente en una base de datos vacía.
data.sql contiene usuarios de prueba.	Sí	Incluye al menos un usuario con rol ADMIN y otro con rol CLIENTE.
El login funciona con datos reales.	Sí	Las credenciales se validan contra los usuarios almacenados en MySQL.

Las páginas privadas no se abren sin sesión.	Sí	Los controladores verifican la existencia de una sesión activa antes de permitir el acceso.
Existe al menos un usuario administrador y uno regular.	Sí	El sistema cuenta con los roles ADMIN y CLIENTE.
El primer CRUD guarda, lista, edita y elimina/desactiva.	Sí	El CRUD de productos permite registrar, listar, editar y desactivar productos.
El código no tiene SQL dentro de los JSP.	Sí	El proyecto utiliza Thymeleaf en lugar de JSP y el acceso a datos se realiza desde los repositorios.
El repositorio tiene commits recientes del equipo.	Por verificar	Se debe revisar el historial de Git y subir los últimos cambios antes de entregar.